

In a Java football club management system, should Player be a normal class or an abstract class if I have StarPlayer and RegularPlayer with different salary calculations? Explain the design trade-offs.

Use an abstract `Player` class.

`Player` should hold the shared data and behavior:

```
</> Java
playerId, fullName, age, nationality,
position, shirtNumber, baseSalary, status
```

Then make the salary calculation abstract:

```
</> Java
public abstract double calculateMonthlySalary();
```

Each subclass provides its own rule:

```
</> Java

public class StarPlayer extends Player {
    @Override
    public double calculateMonthlySalary() {
        return getBaseSalary() * 1.20;
    }
}

public class RegularPlayer extends Player {
    @Override
    public double calculateMonthlySalary() {
        return getBaseSalary() * 1.05;
    }
}
```

This is better than a normal `Player` class because a base `Player` does not have one universal salary formula. Declaring it abstract prevents creating an unclear generic `Player` object and forces each player type to define its own calculation.

A normal class would be simpler if every player used the same salary rule. But with `StarPlayer` and `RegularPlayer`, a normal class often leads to code like:

```
</> Java
if (playerType.equals("Star")) {
    // 20% bonus
} else if (playerType.equals("Regular")) {
    // 5% bonus
}
```

That becomes harder to maintain when more types are added.

With an abstract class and polymorphism, your list can use `ArrayList<Player>` and simply call:

```
</> Java
player.calculateMonthlySalary();
```

Java automatically runs the correct overridden method for `StarPlayer` or `RegularPlayer`.

📄 🌟 🔍 ↻ ...

```
public abstract class Player extends ClubEntity {
    private String fullName;
    private int age;
    private String nationality;
    private String position;
    private int shirtNumber;
    private double baseSalary;
    private String status;

    public class RegularPlayer extends Player {

    }

    public class StarPlayer extends Player {
```